Widgets

Dr. V. Subba Ramaiah

Asst. Prof., Dep. Of CSE

MGIT, Hyd.

What is a widget?

- A **Widget** is a description of part of the user interface.
- Everything in Flutter is a widget.
- Primary component of any flutter application.
- Acts as a UI for the user to interact with the application.
- **building block of the user interface (UI)**
- Think of widgets like **Lego pieces** — each one does a small job, and you combine them to build your full app.

Purpose of a Widget

- Widgets are used to:
- Build the **UI layout**.
- Manage the **state** (data that changes on screen).
- Handle **user interactions** (like button taps).
- Apply **styling and themes**.
- In short: a **widget tells Flutter what to draw** and **how to behave**.

Why we need Widgets

- Without widgets, you'd have to write raw layout code (like Android XML or iOS Storyboards).
  Flutter widgets make UI development:
- **Declarative** (you describe *what* you want, Flutter handles *how*).
- **Faster** (hot reload).
- **Consistent** (same look across Android, iOS, Web).

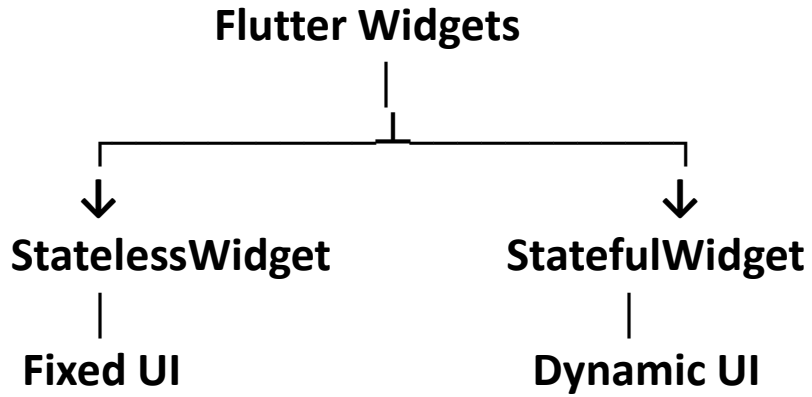
Types of Widgets

- Flutter widgets are mainly divided into **two types:**

Type	Description	When to use
Stateless Widget	UI **does not change** over time	For static content
Stateful Widget	UI **changes** when data or user input changes	For dynamic or interactive screens

Stateful and Stateless Widgets

- A widget is either stateful or stateless. **If a widget can change—when a user interacts with it, for example—it's stateful.**
- **A stateless widget never changes**. Icon , IconButton , and Text are examples of stateless widgets.



Examples

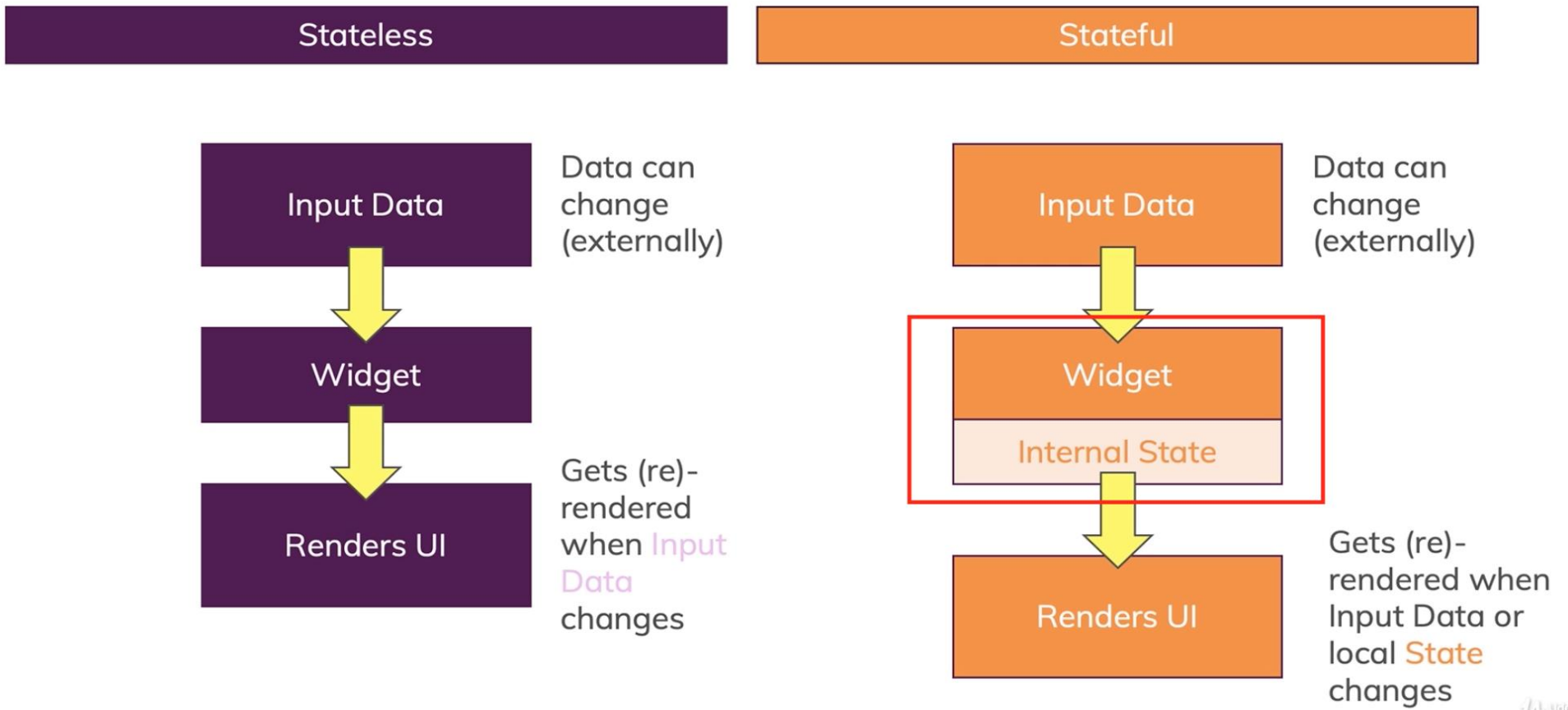
- Stateless widgets are **text, icons, icon buttons, and raised buttons**.
- A stateful widget is dynamic: for example, it can change its appearance in response to events triggered by user interactions or when it receives data. **Checkbox , Radio , Slider , InkWell , Form , and TextField** are examples of stateful widgets.

Feature	StatelessWidget	StatefulWidget
State	No mutable state	Has mutable state
UI changes	Generally based on inputs	Can change during runtime
setState()	No	Yes
Example	Text, Icon, static UI	Counter, Checkbox
Complexity	Simple	More complex
State object	Not separate	Separate State object

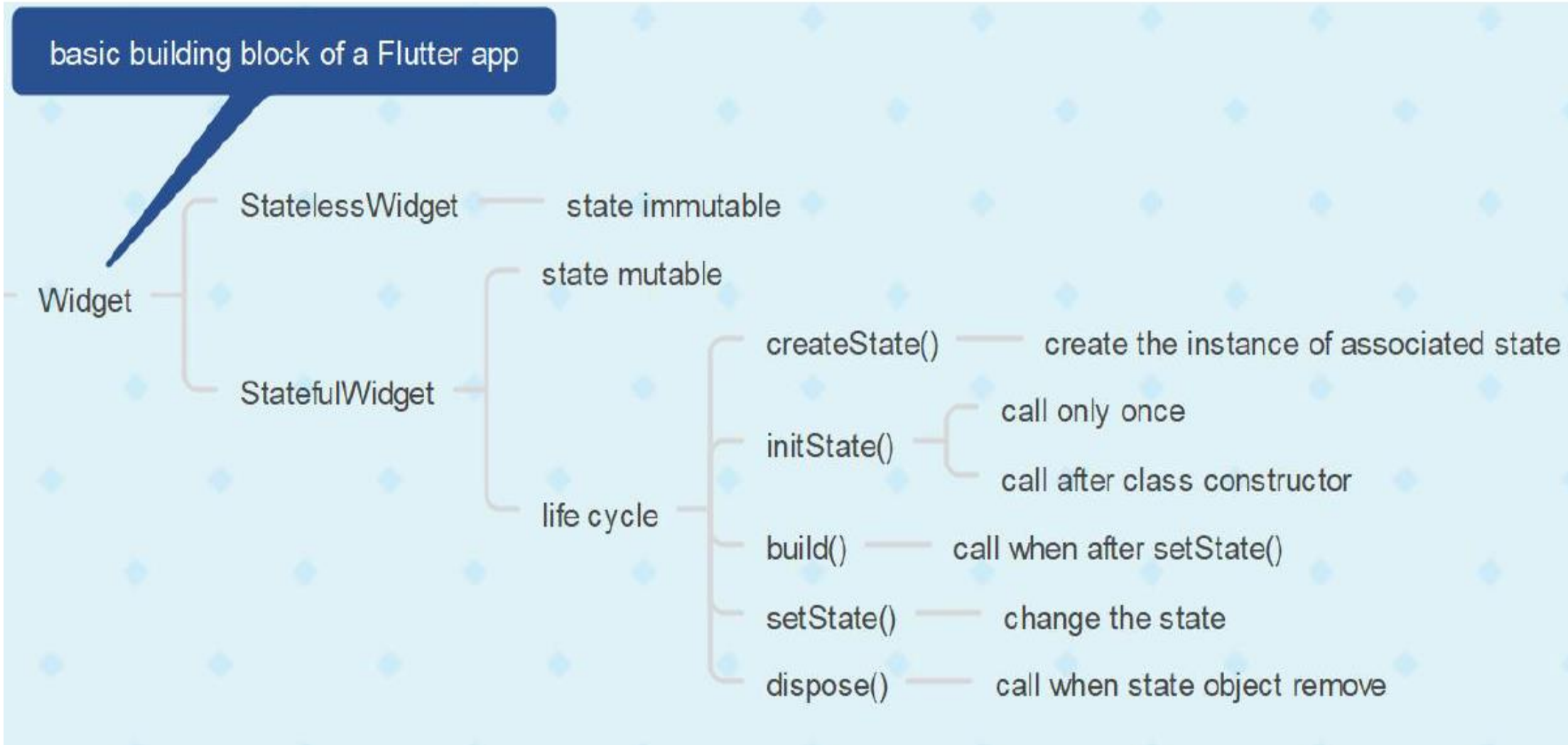
Stateless = UI doesn't manage changing state

Stateful = UI manages changing state

Stateless vs Stateful



Stateful Widget	Stateless Widget
when Widget changes its value, that's Stateful. e.g. Checkbox, Radio button, Textfield	No change in widget value, that's Stateless. e.g. Text, Icon, Icon button, Raised button
Override the CreateState() and return State.	Override the build() and return Widget.
Use when user want to change UI dynamically.	Use when UI remains constant during runtime.
When Widget's state changes, the State object calls setState(), telling framework to redraw widget.	


basic building block of a Flutter app
Widget
StatelessWidget
state immutable
StatefulWidget
state mutable
life cycle
createState()
create the instance of associated state
initState()
call only once
call after class constructor
build()
call when after setState()
setState()
change the state
dispose()
call when state object remove

Basic Structure of a Flutter Application

```
import 'package:flutter/material.dart';
void main() {
runApp(MyApp());
}
```

runApp() takes the root widget and displays it on the screen

packages

1. import 'package:flutter/material.dart';
2. Purpose: Loads the Flutter Material Design library, which contains widgets like MaterialApp, Scaffold, AppBar, Text, etc.

MaterialApp → provides theme and overall app structure.
Scaffold → gives you a full page layout.
AppBar → shows the top bar with title.
Text → displays your content.

MaterialApp

Use / Purpose

- **Main root widget** for Material Design apps.
- Sets up **themes**, **routes**, **navigation**, and **title bar**.
- Provides the base structure and styling (colors, fonts, transitions) following Google's Material Design system.

Why we need

- Without MaterialApp, widgets like Scaffold, AppBar, and TextTheme won't have access to Material styling.
- Gives your app a **consistent look and feel** across platforms.

MaterialApp - Key Properties

Property	Description
`home`	The default screen widget of your app.
`title`	App title (used by Android/iOS task manager).
`theme`	Defines colors, fonts, and styles.
`routes`	Named routes (for navigation).
`debugShowCheckedModeBanner`	Hides debug banner.

```
import 'package:flutter/material.dart';
void main(){
  runApp(MaterialApp(
      home:Text(“MGIT"),
  ));
  }
```

Entry point

```
void main()
{
runApp(MyApp());
}
```

main() is the **starting point of every Dart/Flutter** program.

runApp() tells Flutter which **widget to display first**.

MyApp() is a **custom widget** (your app), defined below.

Scaffold

Use / Purpose

- Provides a **basic visual layout structure** for your screen (page).
- Gives a ready-made framework that includes:
  - App bar
  - Body
  - Floating action button
  - Drawer (side menu)
  - Bottom navigation bar

Why we need

- Without Scaffold, you'd have to manually handle screen layout and safe areas.
- Handles system UI features automatically (like status bar padding, background color, etc.).

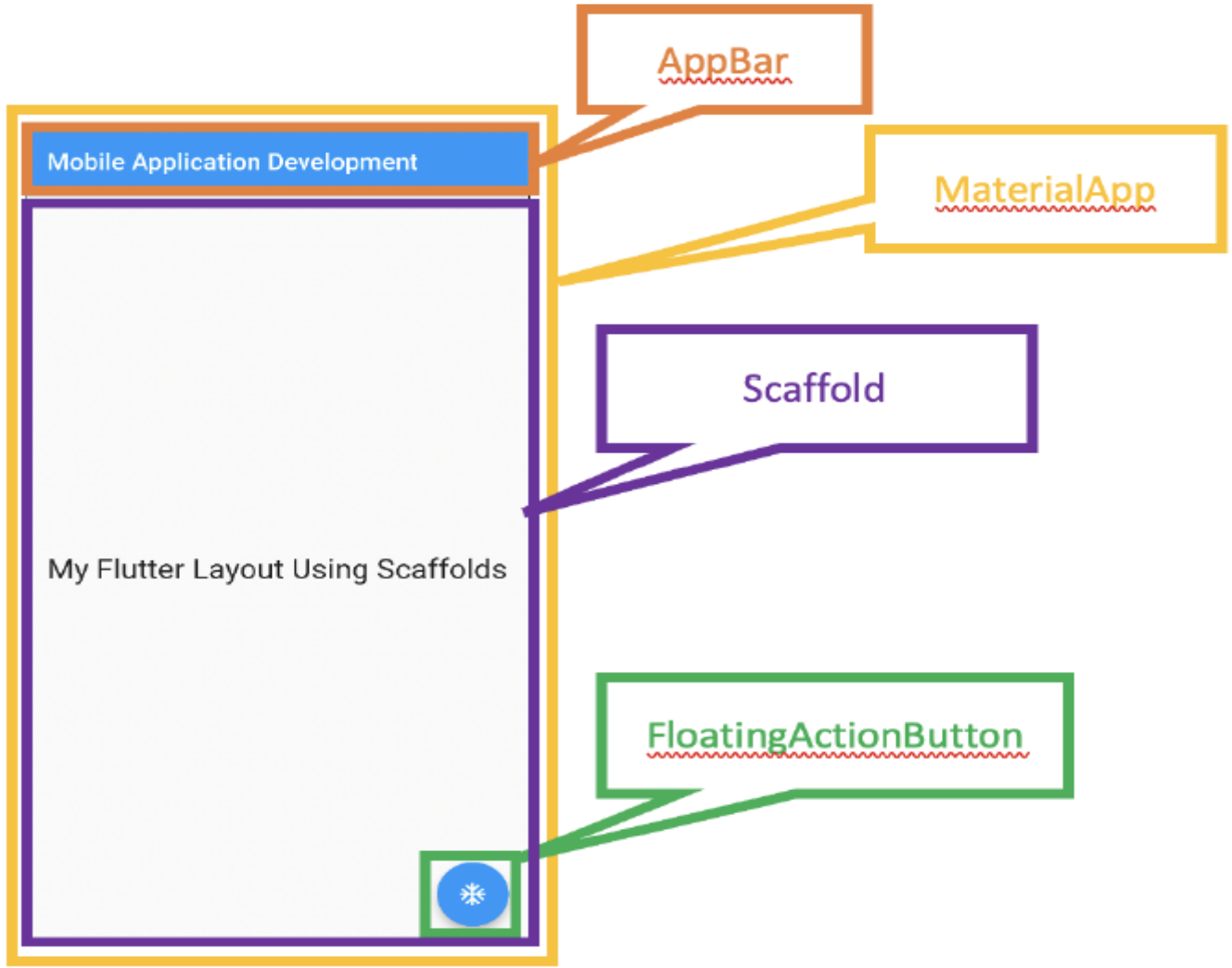


Figure : Three essential parts created in the layout of this app.

Scaffold – Key Properties Property	Purpose
appBar	Adds a top bar (title, icons, actions. etc.)
body	Main screen content of the screen
backgroundColor	Set the background color of the screen
floatingActionButton	Shows a circular button usually used for main action
drawer	Adds a left-side sliding menu
bottomNavigationBar	Adds a bottom bar for navigation
persistentFooterButtons	Adds buttons at the bottom that stay fixed
resizeToAvoidBottomInset	Handles screen resize when keyboard appears

Example

```
Scaffold(
  appBar: AppBar(title: Text('Home')),
  body: Center(child: Text('Welcome!')),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: Icon(Icons.add),
  ),
)
```

AppBar

Use / Purpose

- The **top bar** (header) of your app screen.
- Used to show title, actions (like search, menu, settings), and navigation icons (back button, drawer icon).

Why we need

- Gives a consistent and beautiful top header across all screens.
- Provides built-in animation and system integration.

AppBar – Key Properties

Property	Description
`title`	Widget (usually Text) displayed in the center/left.
`backgroundColor`	Color of the AppBar.
`actions`	List of icons or widgets on the right.
`leading`	Widget on the left (e.g. back button).
`centerTitle`	Center the title (true/false).

Example

```
AppBar(
  title: Text(‘MGIT'),
  backgroundColor: Colors.blueAccent,
  actions: [
    IconButton(onPressed: () {}, icon: Icon(Icons.search)),
  ],
)
```

Text

Use / Purpose

- Displays a **string of text** on the screen.
- One of the simplest but most-used widgets in Flutter.

Why we need

- Every label, title, or message on your screen uses a Text widget.
- Highly customizable for style, font, color, and alignment.

Text – Key Properties

Property	Description
`data`	The text string to display.
`style`	Font size, color, weight, etc.
`textAlign`	Alignment (left, right, center).
`maxLines`	Limit number of visible lines.
`overflow`	Handle overflow text (ellipsis, clip).

Example

```
Text(
  'Welcome to MGIT',
  style: TextStyle(
    fontSize: 22,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
  ),
  textAlign: TextAlign.center,
)
```

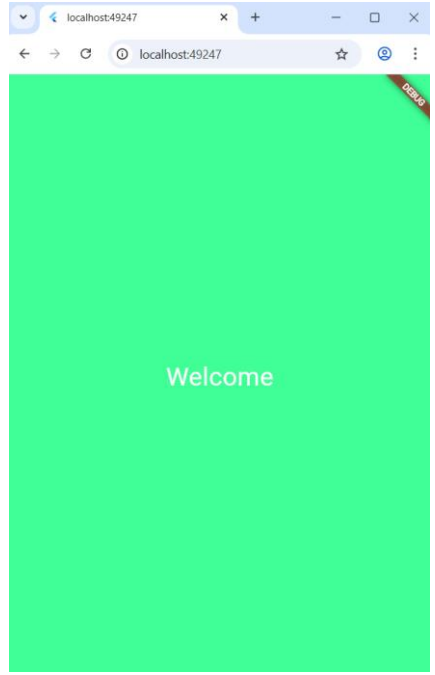
StatelessWidget

```
import 'package:flutter/material.dart';
void main(){
 runApp(const MyApp());}
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text(
            'Welcome to MGIT',
            style: TextStyle(fontSize: 24, color: Colors.purple),
          ),
        ),
      ),
    );
  }
}
```

Doesn’t change once built. Example: labels, icons, static pages.

Output: A simple screen showing “Welcome to MGIT”.
UI stays same always — no data change.

```dart
import 'package:flutter/material.dart';
void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        backgroundColor: Colors.lightBlueAccent, // 👈 sets background color
        body: Center(
          child: Text(
            'Welcome',
            style: TextStyle(fontSize: 30, color: Colors.white),
          ),
        ),
      ),
    );  }}
```



🎨 1️⃣ Using Scaffold.backgroundColor

```
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() =>MyAppState();
}
```

```
class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
        home: Scaffold(
      appBar: AppBar(
        title: Text('Image'),
      ),
      body: Center(
        child: Container(
          height: 200,
          width: 300,
          child: Image.network('img_src'),
        ),
      ),
    ));
  }
}
```